

Capstone Project Packet

"Fidelity Real-Time Portfolio Analytics & Compliance Platform"

Snowflake for Data Engineers · Individual · Self-Paced

How this works

This is **your** build, done **individually** and **outside class time**, at your own pace. It assembles everything from the three days into one cohesive, production-style platform — built **fresh**, in a new database, so you prove you can do it without the course scaffolding.

- **Seven milestones**, each mapping to course material. Do them in order; each builds on the last.
- Every milestone ends with **self-verification queries**. A milestone is DONE when its verifications pass — not when something merely ran. That distinction is most of this job.
- The Day 1–3 lab guides and answer keys are **fair game** as references. Copying a colleague's SQL defeats the only person it could help.
- Estimated effort: **6–10 hours** total.

Rules of the build: everything goes in a new database `FUIDP_CAPSTONE` (so your course `FUIDP` stays intact as a reference), and every script you write is saved to a Git repo (Milestone 7 collects them).

The scenario

Fidelity is standing up a *second* portfolio platform for a newly acquired wealth-management subsidiary. You are the data engineer. The subsidiary's systems deliver:

- **Trade files** arriving continuously (simulate with the Day 1 incremental pattern or in-database inserts)
- **Client/account data** including PII (SSNs, emails) — regulated from day one
- **Market prices** (reuse `FUIDP.STAGING.market_ticks` via a view, or regenerate)

The platform must produce **positions, valuation, and daily P&L**; be **orchestrated** with failure handling and a data-quality gate; be **governed** for four personas before the first analyst logs in; be **performant and cost-fenced**; and be **provably recoverable**.

Milestone 1 — Foundation & Ingestion (*Day 1: Labs 2–6*)

Build:

1. Database FUIDP_CAPSTONE with RAW / STAGING / ANALYTICS schemas; appropriate table types (justify transient vs permanent in comments).
2. Warehouses or reuse: document which workloads share WH_INGEST / WH_ANALYTICS and why.
3. Tables: accounts (with PII columns), securities_master, trades — populated with at least the Day-2 reset script's volumes (generate in-database; adapt the reset script's GENERATOR patterns, don't copy-paste blindly — column names differ on purpose below).
4. **One twist:** the subsidiary's trade feed has a different shape — it includes a commission NUMBER(10,2) column and uses B / S for side instead of BUY / SELL. Your RAW table matches *their* feed; your STAGING layer normalizes it.

Self-verification:

```
SELECT 'raw_trades' AS chk, COUNT(*) AS n, IFF(COUNT(*)>=8000,'PASS','FAIL') AS status
FROM FUIDP_CAPSTONE.RAW.trades
UNION ALL
SELECT 'side_values', COUNT(DISTINCT side), IFF(COUNT(DISTINCT side)=2 AND MAX(side) IN ('S'),
FROM FUIDP_CAPSTONE.RAW.trades
UNION ALL
SELECT 'staging_normalized', COUNT(*),
      IFF(COUNT(*)=0,'PASS','FAIL') -- zero rows with legacy side codes in STAGING
FROM FUIDP_CAPSTONE.STAGING.trades WHERE side NOT IN ('BUY','SELL');
```

Milestone 2 — Incremental Transformation (*Day 2: Labs 8–10, 12*)

Build:

1. A stream on `RAW.trades` and a **task** (or task DAG) that incrementally MERGEs into `ANALYTICS.positions` — accounting for the B / S normalization and **net of commissions** in cost figures.
2. `ANALYTICS.daily_pnl` maintained by an idempotent stored procedure (`CALL` it twice; counts must not change).
3. **Your choice, defended:** rebuild `portfolio_valuation` as either Streams+Tasks or a Dynamic Table — and write 3 sentences in comments justifying the choice using your Day-2 comparison table.

Self-verification:

```
-- Positions must reconcile EXACTLY against a full recompute from raw history:
WITH recompute AS (
  SELECT account_id, symbol, SUM(IFF(side='BUY',quantity,-quantity)) AS q
  FROM FUIDP_CAPSTONE.STAGING.trades GROUP BY 1,2)
SELECT 'positions_reconcile' AS chk,
       IFF(COUNT(*)=0,'PASS','FAIL') AS status
FROM recompute r
FULL OUTER JOIN FUIDP_CAPSTONE.ANALYTICS.positions p USING (account_id, symbol)
WHERE COALESCE(r.q,-999999) <> COALESCE(p.net_quantity,-999999);

-- Idempotency: run your P&L procedure twice for the same date, then:
SELECT 'pnl_idempotent' AS chk,
       IFF(COUNT(*)=COUNT(DISTINCT account_id||'||trade_date),'PASS','FAIL') AS status
FROM FUIDP_CAPSTONE.ANALYTICS.daily_pnl;
```

Milestone 3 — Orchestration with a DQ Gate (Day 2: Lab 11)

Build: An Airflow DAG `fuidp_capstone_pipeline` (reuse the bundle) with at minimum:

`freshness_check` → `transform` → `dq_gate` → `refresh_analytics` → `notify`

The **dq_gate** must enforce three checks of your design (at least one different from the class DAG's three) and demonstrably **fail on bad data**.

Self-verification (evidence, since Airflow is outside SQL): capture two screenshots into your repo

—

1. A fully **green** DAG run (Graph view).
2. A run that **failed at dq_gate** after you inserted a poisoned row — plus the cleanup SQL you wrote, committed to the repo.

```
-- And the gate's checks must currently pass in SQL form:
SELECT 'dq_now' AS chk, IFF(
  (SELECT COUNT(*) FROM FUIDP_CAPSTONE.ANALYTICS.positions) > 0 AND
  (SELECT COUNT(*) FROM FUIDP_CAPSTONE.ANALYTICS.positions WHERE account_id IS NULL) = 0,
  'PASS', 'FAIL') AS status;
```

Milestone 4 — Governance Before First Login (*Day 3: Labs 18–19*)

Build:

1. The two-layer role model: access roles + the four personas (ADVISOR , ANALYST , COMPLIANCE , ENGINEER) scoped to FUIDP_CAPSTONE — **with future grants**.
2. Masking on every PII column (SSN full-mask, email partial, plus the Lab-19 challenge variant: Analyst sees last-4 of SSN).
3. A row-access policy limiting Advisors to their own book via a mapping table.
4. **Tag-based masking** so the next PII column is protected by tagging alone — prove it by adding a phone column and tagging it.

Self-verification:

```
-- Run AS EACH ROLE; record the four results in your write-up:
SELECT CURRENT_ROLE() AS who, account_id, ssn, email
FROM FUIDP_CAPSTONE.RAW.accounts LIMIT 3;
-- PASS = Compliance clear; Analyst ***-**-NNNN; Advisor masked AND row-filtered.

-- Policy inventory must show >= 3 masked columns + 1 row policy:
SELECT POLICY_KIND, COUNT(*) FROM SNOWFLAKE.ACCOUNT_USAGE.POLICY_REFERENCES
WHERE REF_DATABASE_NAME='FUIDP_CAPSTONE' GROUP BY 1;
```

Milestone 5 — Performance & Cost Fence (*Day 2: Labs 13–15*)

Build:

1. Generate RAW.trades_history (≥5M rows via GENERATOR). Choose and apply **one** performance feature (clustering / search optimization / MV) for a workload you define — and

write the 3-line justification: workload shape → feature → expected cost.

2. **Before/after numbers** for that workload (partitions scanned and elapsed) recorded in your write-up.
3. A resource monitor on your working warehouse (quota of your choosing) and `QUERY_TAG='capstone'` on all your sessions from here on.

Self-verification:

```
SELECT 'monitor' AS chk, IFF(COUNT(*)>0,'PASS','FAIL') AS status
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID())) -- run SHOW RESOURCE MONITORS; first
;
-- Simpler: SHOW RESOURCE MONITORS; visually confirm yours, then:
SELECT 'tagged_queries' AS chk, IFF(COUNT(*)>0,'PASS','FAIL') AS status
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TAG='capstone' AND START_TIME>DATEADD('day',-7,CURRENT_TIMESTAMP());
-- (ACCOUNT_USAGE lags ~45 min; INFORMATION_SCHEMA.QUERY_HISTORY() for instant check.)
```

Milestone 6 — Recoverability, Proven (*Day 3: Lab 16*)

Build & prove (commit each statement + result to the repo):

1. Run a destructive mistake of your choice on `ANALYTICS.positions` (WHERE-less UPDATE or DELETE), then recover it via `BEFORE(STATEMENT => <query_id>)`.
2. DROP + UNDROP one table.
3. Create `FUIDP_CAPSTONE_TEST` as a **zero-copy clone** of the whole database, mutate one table in the clone, and show prod unchanged.

Self-verification:

```
-- Post-recovery, positions must STILL reconcile (reuse Milestone 2's reconcile query): PASS re
-- Clone isolation:
SELECT 'clone_isolated' AS chk,
  IFF((SELECT COUNT(*) FROM FUIDP_CAPSTONE.ANALYTICS.positions) <>
    (SELECT COUNT(*) FROM FUIDP_CAPSTONE_TEST.ANALYTICS.positions),'PASS','FAIL') AS status;
```



Milestone 7 — Delivery & Write-Up (Day 3: Lab 17 + course-wide)

Build:

1. A **secure view** exposing valuation **without PII**, granted to a share (add the instructor's consumer account if available; otherwise create the share + grants and document the ADD ACCOUNTS step).
2. **The repo**: every script (DDL, pipeline, policies, DAG, reconciliations), the two Airflow screenshots, and:
3. **A 1-page architecture write-up**: a diagram (ASCII fine), your Streams+Tasks-vs-Dynamic-Table decision, your performance-feature triage with the measured numbers, and one paragraph: *"what I would do differently at 5 billion rows."*

Self-verification:

```
SELECT 'secure_share' AS chk,  
  IFF((SELECT COUNT(*) FROM (SHOW SHARES LIKE 'fuidp_capstone%'))>0, 'PASS', 'FAIL');  
-- And: SELECT * from your secure view AS ROLE ADVISOR -- zero PII columns present at all.
```

Evaluation rubric (100 pts)

Criterion	Pts	What "full marks" looks like
Correctness of analytics	25	Milestone-2 reconcile passes; P&L idempotent; commissions handled
Incremental & idempotent design	20	Stream-driven MERGE; re-runs never duplicate; choice A-vs-B defended
Governance completeness	20	Four-role demo evidence; future grants; tag-masking proven on a new column
Orchestration robustness	15	Green run + a <i>deliberately failed</i> DQ-gate run, both evidenced
Performance & cost awareness	10	Feature matched to workload with measured before/after; monitor + tags live
Recoverability & delivery	10	Recovery proven post-mistake; clone isolated; PII-free share; write-up complete

Submission: repo link (or zip) + the write-up. Verification-query outputs may be pasted as text or screenshots.

Where to get help

Order of operations when stuck: (1) the relevant day's lab guide + answer key; (2) the error message — read it twice, it usually names the fix; (3) Snowflake docs; (4) the instructor, with what you tried. "It doesn't work" is not a question; "the MERGE consumes the stream but positions double on re-run, here's my ON clause" gets answered in minutes.